

数据结构 · 知识手册 17 张卡片 · 约 45 分

一、算法与复杂度

高频 · 必考 / 选择 · 约 6 分钟

时间复杂度怎么判：看最深层循环

判法：只数「最内层基本操作的执行次数」，再看它随 n 的增长阶；常数、低阶项一律丢掉。

- `for(i=1;i<=n;i++)` 单层 $\rightarrow O(n)$
- 内外两层都是 `1..n` $\rightarrow O(n^2)$ ；内层 `j=i` 起步 $\rightarrow$ 仍是 $O(n^2)$ (约 $n^2/2$)
- 内层 `j*=2` 或 `i/=2` $\rightarrow$ 该层 $O(\log n)$
- `while(sum<=n) sum+=i++` $\rightarrow$ 由 $1+2+\dots+i \approx n$ 得 $i \approx \sqrt{n} \rightarrow O(\sqrt{n})$
- 递归式 $T(n)=2T(n/2)+O(n) \rightarrow O(n \log n)$ ； $T(n)=T(n/1)+\dots$ 要看清是几次调用

三个易错

1. 循环变量与 n 无关时，别把 $O(1)$ 的循环算进去（比如固定 26 次、32 次的计数）。
2. 递归的复杂度不等于递归深度：斐波那契朴素递归是 $O(2^n)$ ，深度只有 $O(n)$ 。
3. 空间复杂度算「额外空间」，含递归栈；原地 (in-place) 指辅助空间 $O(1)$ 。

▷ 题目问「最坏情况下」的比较次数时，答案常和平时的平均不同（快排、插入排序都是这样）。

二、线性表

高频 · 必考 / 选择 · 约 6 分钟

顺序表与链表：一句话取舍

操作	顺序表 (数组)	单链表
按位置/下标取元素	$O(1)$	$O(n)$
按值查找	$O(n)$ (有序可折半 $O(\log n)$)	$O(n)$
在第 i 个位置插入	平均移动 $n/2$ 个 $\rightarrow O(n)$	找到第 $i-1$ 个 $O(n)$ ，插入本身 $O(1)$
删除第 i 个	平均移动 $(n-1)/2 \rightarrow O(n)$	已知结点指针时 $O(1)$ (连同找前驱)
空间	需连续、要预估容量、可能浪费	结点分散，另需指针域

记牢

- 链表中「已知指针 p 」能 $O(1)$ 完成的是：在 p 之后插入、删除 p 的后继；删除 p 本身必须找前驱（单链表 $O(n)$ ，双链表 $O(1)$ ）。
- 想同时快速访问首尾结点：循环单链表只设尾指针 $rear$ ，头结点即 $rear \rightarrow next$ ，首尾都是 $O(1)$ 。
- 静态链表用数组模拟， $next$ 存下标，好处是没有指针类型也能做链式操作，常用于 FORTRAN 一类语言与教材题。

▷ 需要「频繁随机访问 + 长度基本不变」→ 顺序表；「频繁插入删除 + 长度变化大」→ 链表。这是选择结构类大题的第一句话。

高频 · 选择 / 计算 · 约 6 分钟

双链表、循环链表与结点数公式

- 双链表：每个结点有 $prior/next$ ，可双向查找；插入 x 到 p 之后的四句指针改动顺序要能默写；删除 p 结点两句即可。
- 循环单链表：判空 $L \rightarrow next == L$ （带头结点）；表尾结点的 $next$ 指回头结点。
- 循环双链表：判空 $head \rightarrow next == head \ \&\& \ head \rightarrow prior == head$ 。
- 结点数与指针域： n 个结点的单链表有 n 个 $next$ 域，其中 $n-1$ 个被用（或 n 个，若成环），空链域 1 个； n 个结点的二叉树有 $2n$ 个孩子域、空链域 $n+1$ 个——这就是线索化的依据。

常考计算：带头结点/不带头结点、表头插 vs 表尾插，最后链表里元素顺序完全相反（头插逆序、尾插保序）。原地逆置单链表就是「把头摘下来逐个用头插法插回新表」。

三、栈与队列

高频·必考 / 计算·约 6 分钟

出入栈合法序列与循环队列

合法序列：入栈序列 $1, 2, \dots, n$ 时，出栈序列必须满足「任一元素后面比它小的元素必须递减」。合法序列总数是卡特兰数 $C(2n, n)/(n+1)$ 。判非法的口号：_out 后面出现的、比它小的数必须是从大到小的连续逆序_。如 3 1 2 4 非法（3 出来后 1、2 只能 2 先 1 后）。

循环队列（数组容量 MaxSize）

- 判空：front == rear；判满（牺牲一个单元）： $(rear+1) \% \text{MaxSize} == \text{front}$ ，此时最多存 $\text{MaxSize}-1$ 个元素。
- 元素个数： $(rear - \text{front} + \text{MaxSize}) \% \text{MaxSize}$ 。
- 想存满 MaxSize 个：另设 size 计数或 tag 标志位，此时判满看 size。
- 入队 $\text{rear}=(\text{rear}+1)\% \text{MaxSize}$ ，出队 $\text{front}=(\text{front}+1)\% \text{MaxSize}$ 。

双端队列：只允许一端出入 → 输出受限；另一端只能进不能出 → 输入受限。受限双端队列能得到的出队序列比栈多、比队列少，常考「哪个序列不可能得到」。

▷ 栈=LIFO、队列=FIFO、递归=隐式栈：把递归写成非递归，本质是自己维护这个栈。

高频·必考 / 大题·约 6 分钟

表达式求值、中缀转后缀

规则：操作数直接输出；运算符与栈顶比较优先级，栈顶优先级 $\geq$ 当前运算符就先出栈输出（左括号例外：入栈后当作最低，只在遇到右括号时才弹出）。

例： $a+b*(c-d)-e/f \rightarrow$ 后缀 $a\ b\ c\ d\ -\ *\ +\ e\ f\ /\ -$ 。

- 后缀表达式求值只用一个栈：遇数入栈，遇符弹出两个（先弹出的是右操作数）算完压回。
- 表达式树的后序遍历 = 后缀式，中序 = 中缀式（加括号后），前序 = 前缀式。
- 优先级： $() > \text{乘除} > \text{加减} > \text{乘方}$ （右结合，处理时要特别注意）。

常考变形：括号匹配（用栈，扫描结束栈空且每个右括号都能配上才合法）；行编辑程序（# 表示退格）；迷宫求解与递归的非递归化都用栈。

四、串与 KMP

高频·必考 / 计算·约 6 分钟

next / nextval 求法与匹配复杂度

408 常用定义（模式串 p 下标从 1 开始）： $next[1] = 0$ ； $next[j] =$ 最大的 $k(1 < k < j)$ ，使得 $p[1..k-1] == p[j-k+1..j-1]$ ，即「前 $j-1$ 个字符组成的串的最长相等前后缀长度 + 1」；找不到就取 1。

手算套路：对每个 j ，看 j 前面那段串的最长「前缀=后缀」长度 L ，则 $next[j] = L + 1$ 。

nextval：若 $p[j] == p[next[j]]$ 则 $nextval[j] = nextval[next[j]]$ ，否则等于 $next[j]$ （把「比较完还要跳一次」的无用跳转提前压缩掉）。

复杂度：求 $next$ 数组 $O(m)$ ，匹配 $O(n)$ ，总体 $O(n+m)$ ；朴素 BF 最坏 $O(n \cdot m)$ 。匹配过程中 KMP 主串指针 i 永不回溯，只移动模式串——这是它和 BF 的本质区别，也是选择题最爱问的一句话。

▷ 常给模式串问 $next$ 数组，务必按题目指定的下标起点（1 起还是 0 起）作答，两套答案差 1，历年争议点就在这里。

五、数组与特殊矩阵

高频·必考 / 计算·约 6 分钟

压缩存储的地址计算

n 阶对称矩阵（下标从 1 起，按行存到下标从 0 起的一维数组）： $a[i][j] (i \geq j)$ 存 $k = i(i-1)/2 + j - 1$ ；上三角元素交换 i, j 后用同一式。共 $n(n+1)/2$ 个元素。下三角（含对角线）同上有 $n(n+1)/2$ ；严格上三角有 $n(n-1)/2$ 。

稀疏矩阵：三元组（行, 列, 值）顺序表——但随机访问和加法不方便；需要频繁增删时用十字链表。

多维数组地址：行优先 $LOC(i, j) = base + ((i-i_0) \cdot \text{列数} + (j-j_0)) \cdot L$ ；列优先把「行数」和「列数」对调。分块/带状矩阵：非零元素分布在 $|i-j| \leq 1$ 的三条对角线上时，每行最多 3 个，压缩后按「前面行里落在带内的元素个数」累加。

易错：题目给的起始下标（0 还是 1）、起始地址（0 还是 100）、每个元素占几字节——三个都要抄下来再算。

六、树与二叉树

高频·必考 / 计算·约 6 分钟

二叉树五条必用性质

1. 第 i 层最多 $2^{(i-1)}$ 个结点；深度 k 的二叉树最多 $2^k - 1$ 个结点。
2. $n_0 = n_2 + 1$ （叶子数 = 度为 2 的结点数 + 1）。总边数 = $n - 1 = \text{度1结点数} + 2 \cdot \text{度2结点数}$ ，两式联立即可解各种「给度数求结点数」的题。
3. 满二叉树：每层都满；完全二叉树：除最后一层外全满，最后一层靠左连续。满 $\subset$ 完全。
4. n 个结点的完全二叉树：深度 $\lfloor \log_2 n \rfloor + 1 = \lfloor \log_2(n+1) \rfloor$ ；结点 i 的双亲 $\lfloor i/2 \rfloor$ 、左孩子 $2i$ 、右孩子 $2i+1$ （超出 n 则为空）。
5. n 个结点的二叉树共有 $n+1$ 个空链域（左右孩子指针合计 $2n$ ，用掉 $n-1$ ）。

结点编号类题的通用解法：先把「度 0/1/2」列表，用 $n_0 = n_2 + 1$ 与 $n = n_0 + n_1 + n_2$ 两式；再按完全二叉树公式定位。

▷ 「已知先序、后序能不能唯一确定二叉树？」——不能，除非每个结点度都清楚（或给出中序）。先+中、后+中、层+中可确定；先+后只能确定度不超过 2 的情况下的形态数，不能唯一还原。

高频·必考 / 大题·约 6 分钟

四种遍历与互推

- 先序 NLR、中序 LNR、后序 LRN、层序（用队列，逐层从左到右）。
- 由先序+中序重建树：先序第一个是根 → 在中序里定位根 → 左边是左子树、右边是右子树 → 递归。
- 由「先序 + 中序」可得后序；由「后序 + 中序」可得先序。层序 + 中序同样能重建（先定根，再按层序逐个归位）。
- 线索二叉树：把空链域改成指向前驱/后继的线索（ltag/rtag 标记是 0 还是线索）。中序线索化后找后继：若 $rtag == 1$ 右线索直接就是；否则找右子树中最左下的结点。中序线索树找前驱对称；先序/后序线索树找前驱或后继往往还得回到根重走——这是常考的「哪种遍历的线索树不能高效双向走」。

树 ↔ 二叉树转换：树的孩子兄弟法转成二叉树后，树的先根 = 二叉树先序，树的后根 = 二叉树中序。森林的先根遍历拼接、层次遍历按队列。

哈夫曼树与前缀编码

构造：每次从森林取两个权值最小的树合并，新根权值为两者之和，重复到只剩一棵。**性质（必须能秒答）**

- n 个叶子 $\rightarrow$ 总结点 $2n-1$ ，非叶结点 $n-1$ ；哈夫曼树里没有度为 1 的结点。
- 带权路径长度 $WPL =$ 所有内部结点权值之和 $= \sum(\text{叶子权值} \times \text{路径长度})$ 。
- 构造过程不唯一（同权可换序），哈夫曼树形态不唯一，但 WPL 唯一且最小。
- 任一字符的编码都不是另一个编码的前缀（前缀编码），因此解码无歧义、不需要分隔符。
- 编码长度 $\neq$ 一定不同：权值大的离根近。定长编码需要 $\lceil \log_2 n \rceil$ 位，哈夫曼编码平均长度 $\leq$ 定长。

常考：给权值求 WPL ；给 WPL 反推某个结点的权值；判断哪棵是哈夫曼树（检查是否每次取最小两个 + 无度 1 结点）。

七、查找结构

高频·必考 / 大题·约 6 分钟

堆、BST、AVL 各自的必考点

堆：完全二叉树 + 大顶（父 $\geq$ 孩子）/小顶。

- 建堆：从最后一个非叶结点 $\lfloor n/2 \rfloor$ 开始向前逐个下沉调整，总时间 $O(n)$ （不是 $O(n \log n)$ ，这是高频陷阱）。
- 插入 $O(\log n)$ 、删除堆顶 $O(\log n)$ （末元素补顶再下沉）。堆排序不稳定， $O(n \log n)$ ，空间 $O(1)$ 。
- 判断是不是堆：按下标逐对比较 $a[i] \geq a[2i]$ 、 $a[i] \geq a[2i+1]$ 。

BST：中序遍历必为递增序列；查找/插入平均 $O(\log n)$ ，退化成单支时 $O(n)$ 。删除结点：叶结点直接删；只有一个孩子则孩子顶替；有左右两子 $\rightarrow$ 用右子树最小（或左子树最大）结点替换，再删那个结点。

AVL：任一结点平衡因子 $\in \{-1, 0, 1\}$ ；插入后自下而上第一个失衡结点做 LL/RR/LR/RL 旋转（LR、RL 是两次单旋）。删除也可能引发 $O(\log n)$ 次调整。常考：插入序列后画 AVL、问旋转类型。

B 树（m 阶）：每结点最多 m 棵子树；除根外至少 $\lfloor m/2 \rfloor$ 棵子树；根至少 2 棵。含 k 个关键字的结点有 k+1 棵子树，关键字数范围 $\lfloor m/2 \rfloor - 1 \sim m-1$ 。所有叶结点（失败结点）在同一层。查找代价 = 磁盘访问次数 $\approx$ 树高，结点内用折半。B+ 树：非叶结点只当索引，全部关键字与数据都在叶结点里，叶结点按关键字大小链成有序链表——所以范围查询顺链走，这是与 B 树最实的区别。散列：装填因子 $\alpha = \text{元素数} / \text{表长}$ ；ASL 只与 α 有关而与表长无关（同一 α 时）。线性探测会「堆积」，ASL 随 α 增大恶化最快；链地址法 α 可以 > 1 。

折半查找与判定树

前提：有序 + 顺序存储（链表不能折半）。

$low=1, high=n, mid=\lfloor (low+high)/2 \rfloor$ （或上取整，题目会指定）；相等命中， $a[mid] > key$ 则 $high=mid-1$ ，否则 $low=mid+1$ ； $low > high$ 时查找失败。

判定树：结点就是比较序列，是一棵平衡二叉树，且任一叶结点（外部结点表示失败区间）的深度只可能相差 1。

- 成功 $ASL \approx \log_2(n+1) - 1$ ；比较次数最多 $\lfloor \log_2 n \rfloor + 1$ 。
- n 个内部结点 $\rightarrow n+1$ 个失败结点。

常考题型

1. 给序列问「第 k 次比较的元素是什么」——直接模拟，别背公式。
2. 问哪棵树可能是折半判定树——检查是否 BST + 是否平衡 + 结点是否按 mid 规则生成。
3. 分块查找：块间有序、块内可无序；表索引 + 块内顺序或折半；最优每块约 $\sqrt{n}$ 个。

八、图

存储结构与两种遍历

存储

- 邻接矩阵： $n \times n$ ，无向图对称、可存于下三角；空间 $O(n^2)$ ，适合稠密图。求度数 $O(n)$ ；判边 $O(1)$ 。 n 个顶点 e 条边的无向图矩阵中有 $2e$ 个 1。
- 邻接表：空间 $O(n+2e)$ （有向为 $n+e$ ，另可建逆邻接表）；适合稀疏图；判边 $O(\text{度})$ 。
- 十字链表（有向）、邻接多重表（无向）：为了「每条边只存一份」而支持高效删边。

遍历 (n 个顶点、 e 条边)

- BFS 用队列：邻接表 $O(n+e)$ ，邻接矩阵 $O(n^2)$ 。能求无权图单源最短路径（按层）。
- DFS 用栈/递归：邻接表 $O(n+e)$ ，邻接矩阵 $O(n^2)$ 。
- 遍历序列不唯一（取决于邻接点存储顺序与起始点）。从某点出发一次 BFS/DFS 只能访问其在连通分量 $\rightarrow$ 无向图连通分量个数 = 「对未访问结点重复启动遍历」的启动次数；有向图的强连通分量要配合原图/逆图两次 DFS（Kosaraju 思路）。
- 生成树边数 $n-1$ ；BFS 生成的是按层展开的树，DFS 生成的树是一条深链形态。

最小生成树、最短路、拓扑与关键路径

最小生成树 (只有连通无向图才有)

- Prim: 从点出发, 每次挑「已选点集到未选点集」的最小边; $O(n^2)$, 适合稠密图。
- Kruskal: 边按权排序, 依次选不构成环的边 (并查集判环); $O(e \log e)$, 适合稀疏图。
- 边权互不相同时 MST 唯一; MST 形态可能不同, 但权值之和唯一, 且是使图连通的最小边集。

最短路径

- Dijkstra: 单源、不能有负权; 每轮确定一个顶点的最短路 $\rightarrow$ 「n 轮、每轮 $O(n)$ 」共 $O(n^2)$ 。
- Floyd: 多源, 可含负权但不得有负权回路; 三重循环 $O(n^3)$, 能处理负边。
- Bellman-Ford / SPFA: 可含负权、能判负环, $O(ne)$ 。
- 无权图最短路径 = BFS 按层。

拓扑排序 (AOV 网): 反复取入度为 0 的顶点; 若还有结点取不出 $\rightarrow$ 图中有环。拓扑序列一般不唯一; 有向图中的环无法拓扑排序。关键路径 (AOE 网): 事件最早发生时间 ve 从源点正推、最迟发生时间 vl 从汇点倒推; 活动 a 的最早开始 = $ve(\text{起点})$, 最迟开始 = $vl(\text{终点}) - \text{活动时间}$, 余量 (时间 slack) 为 0 的活动是关键活动, 从源到汇由关键活动串成的路径就是关键路径, 其长度 = 工程最短工期。

- 关键路径可能有多条; 缩短工期必须压缩关键活动, 且要把所有关键路径一起压 (只压一条无效)。压缩非关键活动不会缩短工期; 关键活动提前完成后, 某些原本非关键的活动可能变成新的关键活动。

九、排序

高频 · 必考 / 选择 · 约 6 分钟

十大排序对照表（背这张就够）

方法	平均	最坏	空间	稳定	关键点
直接插入	$O(n^2)$	$O(n^2)$	$O(1)$	稳	近乎有序时最快，最好 $O(n)$
折半插入	$O(n^2)$	$O(n^2)$	$O(1)$	稳	比较降到 $O(n \log n)$ ，移动仍 $O(n^2)$
希尔排序	$O(n^{1.3})$	$O(n^2)$	$O(1)$	不稳	分组增量排序，一趟不能确定最终位置
冒泡	$O(n^2)$	$O(n^2)$	$O(1)$	稳	每趟至少把一个元素「冒」到最终位置；某趟无交换即完成
简单选择	$O(n^2)$	$O(n^2)$	$O(1)$	不稳	移动次数最少 ($\leq 3(n-1)$)
快速排序	$O(n \log n)$	$O(n^2)$	$O(\log n)$	不稳	每趟把枢轴放到最终位置
堆排序	$O(n \log n)$	$O(n \log n)$	$O(1)$	不稳	建堆 $O(n)$ ，与初始序列无关
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n)$	稳	需要等长辅助空间；2-路归并趟数 $\lceil \log_2 n \rceil$
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(r)$ 或 $O(n)$	稳	不比较关键字，按位分配收集
计数/桶	$O(n+k)$	$O(n+k)$	$O(k)$	稳	要求值域有限

判断口诀

- 不稳定：快、堆、希、选。（想记稳的：插、冒、归、基）
- 每趟结束至少确定一个元素最终位置的：冒泡、简单选择、快排、堆排序。归并、希尔、基数不行。
- 时间性能与初始序列无关的：选择、堆、归并、基数；有关的是插入、冒泡、快排。
- 空间开销：快排要递归栈 $O(\log n)$ （最坏 $O(n)$ ），归并要 $O(n)$ 辅助，其余原地。
- 数据量大且要求 $O(n \log n)$ ：堆排序（稳定最坏）或快排（常数小、平均最快）；链表排序要稳定省空间 → 归并；近乎有序 → 插入。

快排的枢轴、堆的调整与外部排序

快排：一趟从两端向中间扫描（ $low < high$ 时先 **right** 找小于枢轴的填到 **left**，再 **left** 找大于枢轴的填到 **right**），最后枢轴落到相遇位置——枢轴位置之后已确定，左右两半各自递归。

- 最坏出现在「每次枢轴都取到最值」（对已有序数组取端点）→ 退化成 $O(n^2)$ 、递归栈 $O(n)$ 。
改进：三数取中、随机化。
- 递归深度大的那一半后处理可把栈深压到 $O(\log n)$ 。

堆排序（大顶堆）：建堆 → 把堆顶与末尾交换、堆长减 1 → 对新堆顶下沉调整。求「前 k 大」用大小为 k 的小顶堆更省。判断给定序列是不是堆：把下标 $1..n$ 看成完全二叉树逐层比较；不是堆时问「至少调整几个结点」要按下沉顺序模拟。

外部排序：内存装不下时「段内排序 + 段间归并」。

- 初始归并段越多，归并趟数越多；置换-选择能生成更长的初始段（平均约 2 倍内存容量）。
- k 路归并把趟数降到 $\lceil \log_k m \rceil$ （ m 为段数），代价是内存工作区要 k 个缓冲区；用败者树减少内部比较次数。
- 段数不等时按 k 叉哈夫树（最佳归并树）安排归并顺序，使总读写的记录数最少；不足时补长度 0 的虚段，使 $(m-1) \bmod (k-1) == 0$ 。

十、大题套路

高频 · 大题 / 代码 · 约 6 分钟

代码题反复出现的六种解法

408 数据结构代码题（通常 2 题、约 15 分）几乎都落在这几种套路里。先写思路（注释伪代码）再写代码：即使代码有小错，思路分也拿得到。

1. 双指针 / 快慢指针：有序数组去重、找中间结点、找倒数第 k 、判断链表有环（快慢指针相遇）、求两序列公共元素。
2. 原地逆置 + 头插：链表就地逆置、后半段逆置后与前半逐比较（回文链表判定）。
3. 计数/标记数组：值域有限时用 $O(1)$ 辅助数组，如「找未出现的最小正整数」「找主元素的前置统计」。
4. 归并/二分思想：求第 k 小、两有序序列的中位数（各取 mid 比较后丢弃一半， $O(\log n)$ ）。
5. 分块/取模统计：Boyer-Moore 求多数元素（一次撤销式计数 + 二次验证）。
6. 前后缀累积：区间和、最大子段和（Kadane，一次扫描 + 当前和为负就重置）。

答题格式建议（阅卷按点给分）

```
// 思路：先用 1~3 行中文写清算法与边界，再给代码
// 复杂度：时间  $O(\dots)$ 、空间  $O(\dots)$ 
int solve(int a[], int n) { ... }
```

必须自己检查三类边界：空表/长度 1、下标越界、重复元素。这三处也是「代码看着对但扣 2 分」的常见原因。

408 里分值最高的一科，代码大题 2 题约 15 分都出自这科。

「考频」是按题型惯例给出的人工判断，不是官方统计；大纲与分值以当年《考试大纲》《考试分析》为准。

来源：本地手写（依据 408 考试大纲与通行教材整理，不复制任何第三方受版权保护的讲义）；构建于 2026-09-28 10:15。公式渲染失败的条目会以红色等宽字体保留原文。